

Mohammed Rezaan Riyaz

SENIOR FRONTEND ENGINEER

React · Next.js · TypeScript · 7+ years

UAE — open to relocation, onsite & remote

+971-56-618-4561

rezaan6@gmail.com

in linkedin.com/in/rezaan6

github.com/rezaan6

rezaanriyaz.com

Senior Frontend Engineer with 7+ years owning large React codebases from architecture through production. I lead frontend teams, work cross-functionally with design, backend and DevOps, and turn Figma into pixel-perfect, accessible interfaces. I make the structural decisions a codebase has to live with, and take responsibility for how it performs and how safely it ships.

2

platforms architected 0→1
— Hobber, Kodez

-40%

page load time, Kodez CMS

~90%

flow coverage, critical paths

40+

production releases
shipped

EXPERIENCE

Hobber · HQ Abu Dhabi, UAE

JAN 2026 — PRESENT

Senior Frontend Engineer — Lead

MARKETPLACE PLATFORM FOR ENTERTAINMENT, RECREATION, DINING & TOURISM VENDORS

React

TypeScript

Vite

shadcn/ui

Feature-sliced architecture

REST

SignalR

Google Maps API

- Lead frontend development of the Hobber vendor platform, architecting the vendor dashboard from an empty repository in React and TypeScript on a modular, feature-sliced structure.
- Shipped seven core platform modules — authentication, vendor accounts, dashboards, scheduling, payouts, integrations, and team access — as independent slices over one shared component layer built on shadcn/ui and Radix primitives.
- Built the vendor workflows on top: activity management, booking flows, stock scheduling, discount logic, and content management.
- Set the architectural boundaries the codebase is held to, so a new domain is a new folder rather than a refactor, with cross-slice contracts enforced by TypeScript at build time.
- Set up the testing layer from scratch — Vitest for unit and integration, Playwright for end-to-end across the vendor workflows.
- Built real-time vendor messaging on SignalR against the .NET backend, and the location and routing surfaces on the Google Maps API.
- Partner with backend and DevOps engineers on API contracts — pinned against Swagger before building — authentication systems, and platform features.

Axinom · HQ Fürth, Germany

MAR 2024 — NOV 2025

Senior Frontend Engineer — Lead

ENTERPRISE MEDIA TECHNOLOGY DELIVERED ON THE MOSAIC PLATFORM

React

Next.js

TypeScript

GraphQL

SCSS

Shaka Player / DRM

Azure Blob Storage

i18n

Azure DevOps

- Led the frontend team building high-performance web applications for clients including the Goethe-Institut and the Lindau Nobel Laureate Meetings, delivering multiple products 0→1 through full development, testing, and production release cycles.
- Reduced page load time ~15% through bundle optimization, lazy loading, and caching improvements.
- Built on Axinom's Mosaic micro-frontends (Media, Catalogue, Entitlement, DRM) and its APIs for content synchronization and metadata management, rather than hand-rolling media delivery.
- Implemented secure DRM playback with Shaka Player, and designed i18n in from the start for multi-language experiences across regions.
- Delivered large media assets out of Azure Blob Storage through the Azure SDK for the Goethe-Institut build — files big enough that the browser could not simply hold one, so the transfer had to be handled rather than requested.
- Worked directly with designers to translate Figma into pixel-perfect, responsive interfaces, holding SEO and WCAG-aligned accessibility across every customer-facing surface.
- Owned client-facing technical communication — requirements, feasibility, UX trade-offs, and scope — mentored junior engineers, enforced coding standards, and kept CI/CD stable with DevOps across environments.

Senior Frontend Engineer

ENTERPRISE CMS FOR SERVICE MANAGEMENT AND LOGISTICS

React

TypeScript

Storybook

Cypress

Redux

TanStack Query

MUI X Data Grid

Node.js / Express

Sequelize

MySQL

Atomic Design

AWS

- Architected and delivered a React CMS from 0→1 to MVP as a microarchitecture applying SOLID principles, then led 40+ production releases on it.
- Introduced TDD with Cypress and drove automated coverage ~90% on critical paths, gated in CI/CD — which is what let releases stay frequent without a regression freeze.
- Built a Storybook-based reusable component library that cut frontend development time ~30% across projects.
- Reduced page load time ~40% through code-splitting, lazy loading, and asset optimization, with assets served from AWS S3.
- Migrated a legacy Laravel, jQuery and Bootstrap application to React and ExpressJS incrementally without freezing client delivery — and built Node.js BFF layers with the backend team to reduce API latency.
- Built REST integrations against the client's enterprise vendor stack — Amtek (Australia), running on Fiserv, Toshiba, NTT DATA, Park Assist and City systems — across a 250+ table schema.
- Translated Figma designs into pixel-perfect, responsive interfaces; set frontend standards and ran code reviews as the company scaled from 10 to 60+ people.

RaSoft · HQ Colombo, Sri Lanka

DEC 2018 — FEB 2021

Frontend Engineer

SEED-STAGE STARTUP — INTERNAL SYSTEMS AND WEB PRODUCTS

React.js

JavaScript

SCSS

Agile / Scrum

- Joined as a Frontend Engineer Intern in December 2018 and moved into the full role in June 2019 — first end-to-end UI ownership at a seed-stage startup.
- Delivered an Employment Management System from concept to production as the end-to-end UI owner, in React.js, JavaScript, HTML, and SCSS.
- Rebuilt the company website around responsive layout and accessibility, lifting UX and conversion ~15%; the same push contributed to ~20% growth in client acquisition.
- Improved application performance through refactoring and component-level optimization, and ran code reviews and pair-programming sessions to keep the codebase maintainable as it grew.

CORE STACK

REACT & NEXT.JS

- React
- Next.js (App + Pages Router)
- SSR / CSR / ISR
- Hydration
- Suspense
- Error boundaries
- Memoization & render isolation
- Reconciliation

UI, INTERACTION & DESIGN SYSTEMS

- Pixel-perfect Figma implementation
- Design collaboration & handoff
- Interaction & motion design
- Storybook
- Design tokens & theming
- Tailwind CSS
- SCSS
- Material UI
- shadcn/ui
- Radix UI
- Atomic Design

PERFORMANCE

- Core Web Vitals (LCP, CLS, INP)
- Bundle analysis
- Code-splitting
- Lazy loading
- Caching strategies
- Lighthouse

SECURITY

- XSS prevention
- Secure authentication flows
- Token handling
- Content validation
- Secure API integration patterns
- Dependency risk awareness

AI-ASSISTED DELIVERY

- Agent orchestration
- MCP integrations
- Synthetic test data
- Automated PR & deploy workflows

LANGUAGE & FRAMEWORKS

- TypeScript
- JavaScript
- Zod
- Typed API boundaries
- Angular (legacy systems)

STATE & DATA

- TanStack Query
- Redux Toolkit
- Zustand
- React Hook Form
- REST
- GraphQL
- Algolia search
- Cache invalidation

TESTING & QUALITY

- Vitest
- Playwright
- Jest
- Cypress
- TDD
- E2E & integration
- CI quality gates
- WCAG accessibility
- BrowserStack

BACKEND & DELIVERY

- Node.js
- ExpressJS (BFF)
- PHP / Laravel
- MySQL
- Sequelize
- MongoDB
- AWS S3
- Docker
- GitHub Actions
- Vercel
- Azure Blob Storage
- Jira
- Azure DevOps

EDUCATION

BSc (Hons) Computer Science & Software Engineering — First Class

University of Bedfordshire · 2022

Dissertation: BlockVote — blockchain-based e-voting on Ethereum (Solidity, Truffle, React, MetaMask)

Pearson BTEC Level 5 HND — Computing (Software Engineering)

British College of Applied Studies · 2020

Higher Diploma — Mechatronics Engineering

ICBT, moderated by Cardiff Metropolitan University · 2019

MEASUREMENT NOTES

What each headline figure is, and where it stops. Counts are checkable, readings depend on conditions, estimates are judgements — they are not the same kind of claim and are not presented as though they were.

2 platforms architected 0→1 — Hobber, Kodez · Count

The Hobber vendor platform and the Kodez CMS, each architected from an empty repository through to production.

~90% flow coverage, critical paths · Before/after reading

Cypress coverage of the critical paths — authentication and permissions, anything that writes to the database, and every enterprise integration surface. Flow coverage on those paths, not whole-repo line coverage. The number deliberately excludes the parts of the codebase that were cheaper to cover lower down the pyramid.

–40% page load time, Kodez CMS · Before/after reading

A before/after page-load read on the Kodez CMS, taken against the app as it stood before the code-splitting, lazy-loading and asset work. Approximate, and a lab reading rather than real-user field data. Several changes shipped across that period, so it is a contribution to the improvement rather than the sole cause.

40+ production releases shipped · Count

Production releases shipped on the Kodez CMS across three years — features, defect fixes and database work.